

Warsaw University of Technology

FACULTY OF
ELECTRONICS AND INFORMATION TECHNOLOGY



Institute of Radioelektroniki i Technik Multimedialnych

Final project

Studia Podyplomowe

Głębokie Sieci Neuronowe - Zastosowania w Mediach Cyfrowych

Grafowe sieci neuronowe w przewidywaniu zdarzeń klinicznych

Marta, Monika

student record book number XXXX

thesis supervisor

YYYYY

WARSAW 2026

Grafowe sieci neuronowe w przewidywaniu zdarzeń klinicznych

Streszczenie.

Głównym celem pracy jest Nacisk w pracy został położony na ... W pierwszej części pracy przedstawiono ... Następnie opisano ... Druga część pracy rozpoczyna się od ... Eksperymenty opisane w dalszej części pracy przedstawiają ... Dodatkowym etapem pracy jest ... W zakończeniu pracy przedstawiono podsumowanie wykonanych działań oraz dalsze działania, które mogą przynieść poprawę ...

Słowa kluczowe: NFT, ChatGPT, Buzzword300

Graph neural networks in predicting clinical events

Abstract.

Summary in English.....

Keywords: ChatGPT, Buzzword300



.....
miejscowość i data
place and date

.....
imię i nazwisko studenta
name and surname of the student

.....
numer albumu
student record book number

.....
kierunek studiów
field of study

OŚWIADCZENIE

DECLARATION

Świadomy/-a odpowiedzialności karnej za składanie fałszywych zeznań oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie, pod opieką kierującego pracą dyplomową.

Under the penalty of perjury, I hereby certify that I wrote my diploma thesis on my own, under the guidance of the thesis supervisor.

Jednocześnie oświadczam, że:
I also declare that:

- niniejsza praca dyplomowa nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (Dz.U. z 2006 r. Nr 90, poz. 631 z późn. zm.) oraz dóbr osobistych chronionych prawem cywilnym,
- *this diploma thesis does not constitute infringement of copyright following the act of 4 February 1994 on copyright and related rights (Journal of Acts of 2006 no. 90, item 631 with further amendments) or personal rights protected under the civil law,*
- niniejsza praca dyplomowa nie zawiera danych i informacji, które uzyskałem/-am w sposób niedozwolony,
- *the diploma thesis does not contain data or information acquired in an illegal way,*
- niniejsza praca dyplomowa nie była wcześniej podstawą żadnej innej urzędowej procedury związanej z nadawaniem dyplomów lub tytułów zawodowych,
- *the diploma thesis has never been the basis of any other official proceedings leading to the award of diplomas or professional degrees,*
- wszystkie informacje umieszczone w niniejszej pracy, uzyskane ze źródeł pisanych i elektronicznych, zostały udokumentowane w wykazie literatury odpowiednimi odnośnikami,
- *all information included in the diploma thesis, derived from printed and electronic sources, has been documented with relevant references in the literature section,*
- znam regulacje prawne Politechniki Warszawskiej w sprawie zarządzania prawami autorskimi i prawami pokrewnymi, prawami własności przemysłowej oraz zasadami komercjalizacji.
- *I am aware of the regulations at Warsaw University of Technology on management of copyright and related rights, industrial property rights and commercialisation.*



Oświadczam, że treść pracy dyplomowej w wersji drukowanej, treść pracy dyplomowej zawartej na nośniku elektronicznym (płyce kompaktowej) oraz treść pracy dyplomowej w module APD systemu USOS są identyczne.

I certify that the content of the printed version of the diploma thesis, the content of the electronic version of the diploma thesis (on a CD) and the content of the diploma thesis in the Archive of Diploma Theses (APD module) of the USOS system are identical.

.....
czytelny podpis studenta
legible signature of the student

Contents

List of Symbols and Abbreviations	9
1. Introduction	10
2. Main objective	11
3. Related work	12
3.1. Architecture	13
3.1.1. RGCN	13
3.1.2. Transformer	13
3.2. HCR	14
3.2.1. Metrics	15
4. Dataset	16
4.1. Synthetic dataset	16
4.1.1. Knowledge graph	16
4.1.2. Patient data	19
4.1.3. Evaluation scenarios	20
4.1.4. Resulting patient-level dataset	22
4.2. Benchmark Dataset	24
4.2.1. Last-FM dataset	24
4.2.2. Hetionet knowledge graph	24
5. Main Framework	25
5.1. Link prediction in Graph Reconstruction	26
5.2. Endpoint prediction	26
5.2.1. Data Representation	26
5.2.2. Model architecture	27
5.2.3. HCR application	29
5.2.4. Training setup	30
6. Experiments results	31
6.1. Link prediction in graph reconstuction	31
6.2. Endpoint prediction	32
6.2.1. Architecture	32
6.2.2. HCR configuration	35
6.2.3. Results per endpoint	38
7. Models benchmark	39
7.1. Link prediction	40
7.2. Endpoint prediction	41
7.2.1. Data Representation	41
7.2.2. Results	41
8. Conclusions	45

References	51
Spis rysunków	52
Spis tabel	53

List of Symbols and Abbreviations

GNN – Graph Neural Network

DAG – Directed Acyclic Graph

HCR – Hierarchical Correlation Reconstruction

ADR – Adverse Drug Reaction

AKI – Acute Kidney Injury

DILI – Drug-Induced Liver Injury

GAT – Graph Attention Network

SAGE – Graph SAGE

RGCN – Relational Graph Convolutional Network

AUC – Area Under the Curve

AUPRC – Area Under the Precision-Recall Curve

1. Introduction

Krótki opis dziedziny problemu.

Motywacja, dlaczego warto zająć się rozwiązaniem postawionego problemu.

Opis wkładu własnego - co zrobiono w ramach przygotowywania pracy dyplomowej.

Np. przygotowanie zbioru danych, opracowanie i implementacja architektury modelu itp.

YOLO **redmon2016you**

2. Main objective

The aim of this thesis is to develop a graph neural network models that could be applied in two tasks: a) to predict the presence of hidden links in a causal graph, b) to predict the values of endpoints for entities based on tabular data represented as a causal graph.

The two tasks are complementary ... [we will add here how the two tasks relate to each other in medicine + picture that shows it] The models will be trained and evaluated on synthetic datasets, as well as on real-world datasets. The performance of the models will be compared with baseline methods, and the results will be analyzed to understand the strengths and limitations of the proposed approach.

3. Related work

Przegląd literatury związanej z rozwiązywanym problemem.

W pracy **he2016deep** opisano W **liu2022convnet** zaproponowano nowatorską architekturę.... Przykładowy rysunek 3.1.

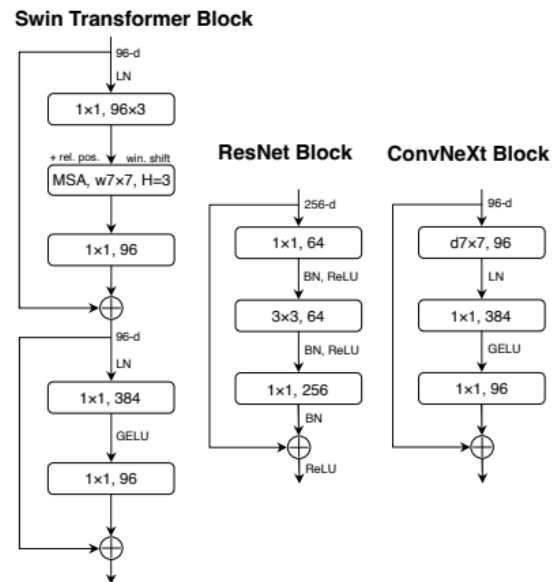


Figure 3.1. Porównanie bloków rezydualnych wykorzystywanych w sieci Swin Transformer, ResNet i ConvNeXt.

3.1. Architecture

3.1.1. RGCN

3.1.2. Transformer

3. Related work

3.2. HCR

Endpoint prediction is a node classification tasks in data.

3.2.1. Metrics

Residual connections switch the effective per-layer propagation operator from the (nilpotent, on a DAG without self-loops) adjacency operator, which is the standard formalization of the over-smoothing regime and is the mechanism through which multi-layer signal survives on a directed acyclic structure.

PairNorm (Zhao et Akoglu, 2020) re-centers and re-scales node representations after each layer to keep the pairwise distance between representations from collapsing, without preventing the underlying mixing.

DropEdge (Rong et al., 2020) randomly removes a fraction of edges independently in every layer during training, slowing the rate of information mixing and acting as a structural regularizer.

Jumping Knowledge (Xu et al., 2018) aggregates the outputs of all layers (via concatenation, cat, or elementwise max) rather than only the last one, letting a node fall back on a shallower, less-smoothed representation when the deepest layer has become uninformative.

Lift is the AUPRC divided by the prevalence baseline, that is, the factor by which the model improves over a random ranking; at a mean prevalence of 5.5% a lift of 2.3 corresponds to an AUPRC of 0.125. *% ceiling* is defined as

$$\frac{\text{AUC} - 0.5}{\text{AUC}_{\text{Bayes}} - 0.5} \cdot 100\%, \quad (1)$$

where $\text{AUC}_{\text{Bayes}} = 0.651$ is the Bayes ceiling averaged over the same eight endpoints. It expresses the share of attainable signal recovered by the model. Both terms are subtracted by 0.5 so that a random classifier scores 0%. Unless stated otherwise, the epoch is selected on the validation set and test metrics are reported from that epoch.

4. Dataset

Deep learning research on medical knowledge graphs currently leaves two related gaps unaddressed. First, causal modeling is rarely applied at the level of the full patient trajectory. Second, most knowledge graphs encode relationships extracted from literature, without integrating patient-level empirical data on how outcomes are actually observed (as well noted by Toonsi et al. [1]). To address both gaps, we generated a synthetic dataset that jointly encodes a known causal structure and its patient-level empirical realizations, providing ground truth for both link prediction and node classification. This gave a full control over how knowledge is encoded in the graph, avoiding reconstruction from a partially observed real-world dataset. The generation process, the resulting graph, and the derived patient data are described in detail in this chapter.

Additionally, there is included a description of benchmark datasets used for testing the prototyped models.

4.1. Synthetic dataset

The dataset includes two structures:

1. a causal knowledge graph (DAG) that encodes causal patient pathways from patient characteristics and drug exposures to clinical events (endpoints);
2. an empirical dataset — a patient-level table with each patient’s characteristics and observations, generated from the knowledge graph (Section 4.1.2).

4.1.1. Knowledge graph

The construction of the knowledge graph started from defining the clinical endpoints to model and predict, and identifying which patient characteristics and drug exposures are known to influence them. This approach – starting from the target and working backwards to its determinants – was chosen so that every path in the graph corresponds to a patient clinical pathway.

As our team includes a team member with a pharmacology background, her domain knowledge was used for constructing and verifying the initial version of the knowledge graph – the first set of endpoints, risk factors, and drug classes. Candidate mechanistic pathways connecting a drug exposure to a given endpoint, including specific drug-drug and drug-disease interaction gates, were proposed with the support of Fable¹. All proposals generated with its support were accepted, rejected, or modified by the team member with a pharmacology background before being incorporated into the graph specification.

The graph was built iteratively. An initial, smaller version of the DAG covered a limited set of drug classes, mechanisms, and endpoints. Once this core structure was validated, it was progressively extended with additional nodes and edges. The final version includes

¹ Fable refers to Claude Fable 5, a large language model developed by Anthropic, which was used as an AI-assisted modelling aid to draft candidate mechanistic pathways for review.

155 nodes organised into six layers specified in Tables 4.1 and 405 directed edges grouped into categories in 4.2.

Tabela 4.1. Node types in the patient causal DAG.

Node type	Description
patient_context	Patient-level covariates (e.g. demographics, comorbidities).
drug_exposure	Drugs the patient is exposed to.
mechanism	Intermediate physiological mechanisms mediating the effect of a drug or context on further stages.
adr_or_intermediate_state	Intermediate adverse or physiological states positioned between mechanisms and clinical endpoints.
observation_or_selection	Nodes representing observation, documentation, or selection processes rather.
clinical_endpoint	Target adverse clinical outcomes (e.g. AKI, DILI, Hospitalization).

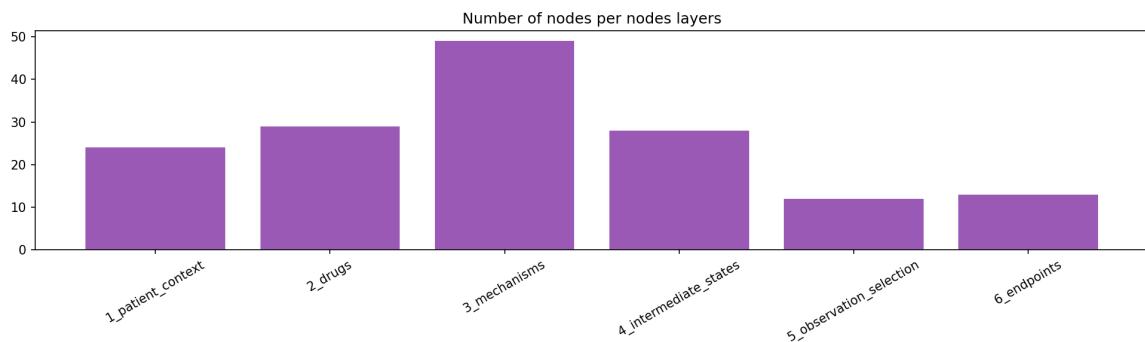


Figure 4.1. Frequency of nodes types.

Beyond its topology, every node in the graph carries a set of attributes which are used directly in the data-generating process described in Section 4.1.2:

- `base_prevalence` encodes a population-level probability of a node being present in a patient who has none of the node's risk factors active.
- `severity_weight` encodes the clinical severity assigned to a node, independent of how frequently it occurs.
- `observability` encodes how reliably a true clinical state is expected to be recorded in practice.

As edges bring different information based on edge type, they were assigned declared attributes, such as:

Tabela 4.2. Semantic categories of directed edges in the patient causal DAG.

Category	Included edge types
Background risk and treatment assignment	risk_modifier, confounding, latent_confounding, treatment_indication, treatment_burden
Drug effects and mechanistic pathways	drug_to_mechanism, causal_mechanistic, mechanism_to_adr, adr_to_intermediate
Clinical outcome progression	adr_to_endpoint, endpoint_to_hospitalization
Observation and selection processes	observation, selection, observation_process
Polypharmacy, aggregate burden, and interactions	drug_to_burden, drug_to_load, interaction_gate_component, drug_pairwise_gate, drug_triple_gate, interaction_amplify, adr_burden_component, burden_threshold

- `effect_size` encodes the strength of the causal effect of the source node on the target node.
- `effect_sign` encodes the direction of that effect (risk-increasing or risk-decreasing).

In fig. 4.2 it is visualized an overview of DAG graph.

The final version of the DAG was verified statistically with the metrics reported in table 4.3. They confirm the graph's acyclicity and characterize its structural complexity for modelling purposes.

Tabela 4.3. Structural statistics of the causal knowledge graph.

Metric	Value
Nodes / edges	155 / 405
Graph density	0.017
Acyclicity	True
Self-loops / duplicate edges	0 / 0
Source nodes / sink nodes	9 / 12
Direct parents per clinical endpoint	mean 4.7 (range 1–14)
Ancestors per clinical endpoint	median 31 (range 9–126)
Longest root-to-endpoint path	median 12 edges (range 5–16)
Shortest root-to-endpoint path	1 edge for 10 of 13 endpoints; 3–4 edges for the three rarest endpoints
Drug–endpoint pairs sharing a common ancestor	85

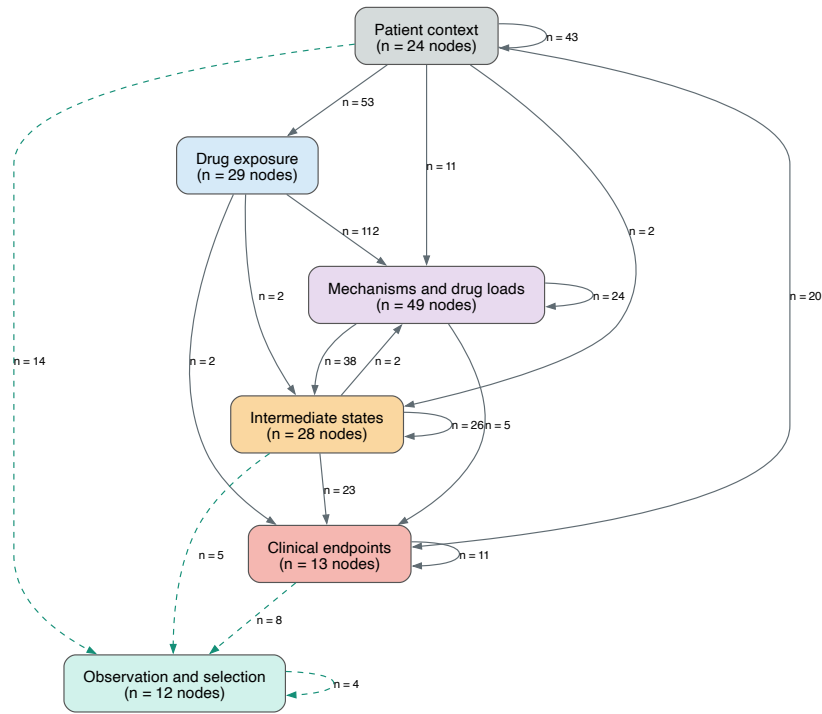


Figure 4.2. DAG visualization

4.1.2. Patient data

The patient-level data generation procedure was designed to produce tabular records that provide empirical evidence for knowledge-graph reconstruction and downstream prediction of clinical outcomes. Its purpose was to transform the static causal DAG specification into a patient-level dataset in which each individual is represented by a realised configuration of exposures, clinical states, and outcomes. This enables the graph representation to be used as a dynamic, data-supported structure for multiple downstream tasks. For each of 20,000 simulated patients, the generator assigns a realised value to each of the 155 nodes in the causal DAG. Values are generated in topological order, following the causal direction of the graph. For every patient, upstream context variables are generated first, followed by drug exposures, physiological mechanisms, and intermediate adverse states. Clinical endpoints are generated only after all their parent variables have been realised. Each node value is generated by one of three mechanisms: a stochastic process, a deterministic computation of other nodes, or a deterministic observation rules. The following paragraphs describe each mechanism.

Stochastic nodes Most nodes – patient covariates, drug exposures, intermediate mechanisms, and clinical endpoints – are generated as Bernoulli draws whose probability is a logistic function of their parents:

$$\eta_i = \text{logit}(p_i^0) + \sum_{j \in \text{parents}(i)} s_{ij} \cdot w_{ij} \cdot x_j, \quad x_i \sim \text{Bernoulli}(\sigma(\eta_i)), \quad (2)$$

where p_i^0 is the node's baseline prevalence, x_j is the realised value of parent j , and w_{ij} , s_{ij} are the effect size and effect sign declared on the edge $j \rightarrow i$. Nodes are generated in topological order, so that every parent has already been assigned a value before its children are processed.

Three conditions affect the implementation:

- A parent with more than two distinct realised values (i.e. a count or continuous node, such as `polypharmacy_burden`) is standardised before entering the sum, $(x_j - \bar{x}_j)/\text{sd}(x_j)$, so that its contribution to η_i is on a comparable scale to binary parents.
- p_i^0 is taken from the `base_prevalence` column of the node specification².
- If missing, `effect_size`, and `effect_sign` defaults to 0.5 and +1 accordingly.

Deterministic derived variables. Some variables are derived deterministically from values that have already been generated based on patterns of polypharmacy, drug interactions, and adverse-event burden explicit in the graph.

Table 4.4 summarises the deterministic variables and their generating rules.

Deterministic observation rules A small set of variables is generated deterministically as the product of an underlying clinical state and its associated observation process. For example,

```
elevated_creatinine_recorded = creatinine_rise × creatinine_tested.
```

These variables represent recorded clinical evidence rather than the underlying patient state alone. Thus, a creatinine rise may be present but remain absent from the recorded data if no creatinine test is performed. Analogous deterministic rules are applied to recorded ALT/AST elevation and recorded QT prolongation.

The observation variables distinguish clinical truth from documentation and are retained as part of the complete data-generating process. They also provide a structured representation of testing and recording mechanisms that is relevant to the construction and interpretation of alternative observational scenarios.

4.1.3. Evaluation scenarios

Data was generated in five controlled scenarios constructed to evaluate model robustness under common challenges of observational clinical data. All scenarios use the

² If base prevalence value is missing, a type-dependent default is used instead (0.16 for drug exposures, 0.02 for clinical endpoints, 0.35 for observation/selection nodes, 0.10 otherwise)

Tabela 4.4. Deterministic variables in the synthetic patient generator.

Variable group	Rule type	Generating rule
Renal, CNS, bleeding, serotonergic, and hepatic ddi_* gates	Conjunctive gate	A binary variable equal to 1 only when all listed drug-exposure components are active. For example, $ddi_renal_triple_whammy = nsaid \wedge acei_arb \wedge diuretic$.
Drug-disease interaction gates (drug_disease_*)	Conjunctive gate	A binary variable equal to 1 when the relevant drug exposure or drug load and the associated patient risk factor are both active.
active_drug_count	Drug count	Sum of all 29 binary drug_exposure variables.
Drug-load variables (nephrotoxin_load, hepatic_drug_load, qt_drug_load_v3, cns_depressant_load, bleeding_risk_load, serotonergic_load, cyp_inhibitor_load)	Drug count	Sum of active drug exposures belonging to a predefined clinical risk group.
ddi_qt_multidrug_load	Drug count	Sum of active QT-prolonging drug exposures specified in the generator.
ddi_serotonergic_high_load	Threshold	Equal to 1 when serotonergic_load is at least 2.
cumulative_adr_burden	Weighted sum	Severity-weighted sum of intermediate adverse states plus $0.08 \times active_drug_count$.
severe_adr_burden	Threshold	Equal to 1 when cumulative_adr_burden is at least 4.0.

same underlying synthetic pharmacotherapy DAG and preserve the semantic meaning of node and edge types. Each scenario modifies either the available information, the sampling mechanism, the treatment-assignment process, the observation process, or the institutional context.

For example, the `selectionbias` scenario restricts the cohort to patients with hospital contact, creatinine testing, and liver-function testing. Consequently, inclusion depends on downstream healthcare-contact and diagnostic processes. The observation and documentation variables therefore support interpretation of this scenario.

All of scenarios are presented in Table ??.

Tabela 4.5. Synthetic evaluation scenarios and their intended data challenge.

Scenario	Primary purpose
<code>clean</code>	Reference setting with complete generated information.
<code>hidden_confounder</code>	Unmeasured confounding: <code>unobserved_severity</code> remains active in the generator but is withheld from the observed data.
<code>selection_bias</code>	Selection bias induced by restricting the cohort to patients with hospital contact, creatinine testing, and liver-function testing.
<code>no_overlap</code>	Positivity violation caused by subgroup-specific treatment probabilities.
<code>noisy_documentation</code>	Imperfect clinical recording, including false negatives and a 1% false-positive recording probability.
<code>multihospital</code>	Institutional heterogeneity in treatment assignment and documentation processes across four synthetic hospitals.

4.1.4. Resulting patient-level dataset

The final dataset generated contains 20,000 simulated patients for every scenario described. Each row represents one patient and each graph node is represented by a corresponding patient-level variable. Table 4.6 summarises the structure and value types of the resulting dataset. A complete variable-level data dictionary is provided in Appendix 8.

Clinical endpoints

The 13 clinical endpoints are binary stochastic variables generated after their direct parents have been realised. Their values are the ground-truth labels used for endpoint prediction. Table 4.7 lists the endpoint names and their direct parents in the final DAG.

Patient-level data splits Patients were partitioned into mutually exclusive training, validation, and test sets in proportions of 70%, 15%, and 15%, respectively. For the `clean` scenario containing 20,000 patients, this corresponds to 14,000 training patients, 3,000 validation patients, and 3,000 test patients.

Splits were created at the `patient_id` level using a deterministic greedy multi-label balancing procedure with seed 20260722. Patients with rare endpoint combinations were

Tabela 4.6. Schema of the generated patient-level dataset.

Variable group	Number	Value type
patient_id	1	Integer identifier.
hospital_id	1	Categorical integer identifier.
patient_context	24	Mostly binary; selected count and baseline-measurement variables.
drug_exposure	29	Binary.
mechanism	49	Binary, count, and deterministic gate variables.
adr_or_intermediate_state	28	Mostly binary; derived score and threshold variables.
observation_or_selection	12	Binary.
clinical_endpoint	13	Binary.

Tabela 4.7. Endpoint prevalence and AUROC ceiling in the evaluation subset.

Endpoint	Positive cases	Prevalence (%)	AUROC ceiling (%)	Parents
AKI	393	13.10	81.13	6
DILI	55	1.83	65.29	7
Delirium	111	3.70	66.80	4
Depression	254	8.47	65.25	6
Falls	234	7.80	69.67	7
GI bleeding	64	2.13	57.04	5
Hospitalization	676	22.53	75.07	14
Hyperkalemia	146	4.87	58.98	2
Hyponatremia	236	7.87	63.83	3
QT arrhythmia	48	1.60	62.24	4
Serotonin syndrome	8	0.27	55.45	1
Rhabdomyolysis	11	0.37	53.02	1
Lactic acidosis	17	0.57	52.32	1

4. Dataset

assigned first, and assignments were chosen to preserve the prevalence of each clinical endpoint across the three splits. Consequently, every patient appears in exactly one split, while the multi-label endpoint distribution remains approximately balanced.

A single master split assignment was derived from the clean scenario and reused across all alternative scenarios. This ensures that performance differences between scenarios reflect the intended changes in data scenarios.

4.2. Benchmark Dataset

4.2.1. Last-FM dataset

4.2.2. Hetionet knowledge graph

Hetionet is a heterogeneous network (*hetnet*) that integrates biomedical knowledge from 29 publicly available resources. It represents biological and pharmacological entities as nodes and documented relationships between them as typed edges.

Hetionet v1.0 contains 47,031 nodes belonging to 11 node types and 2,250,197 relationships of 24 types. The graph includes compounds, diseases, genes, anatomies, pathways, biological processes, molecular functions, cellular components, pharmacologic classes, side effects, and symptoms. This heterogeneous structure makes Hetionet suitable for node classification task described further. The details of benchmark is included in Section 7.2.

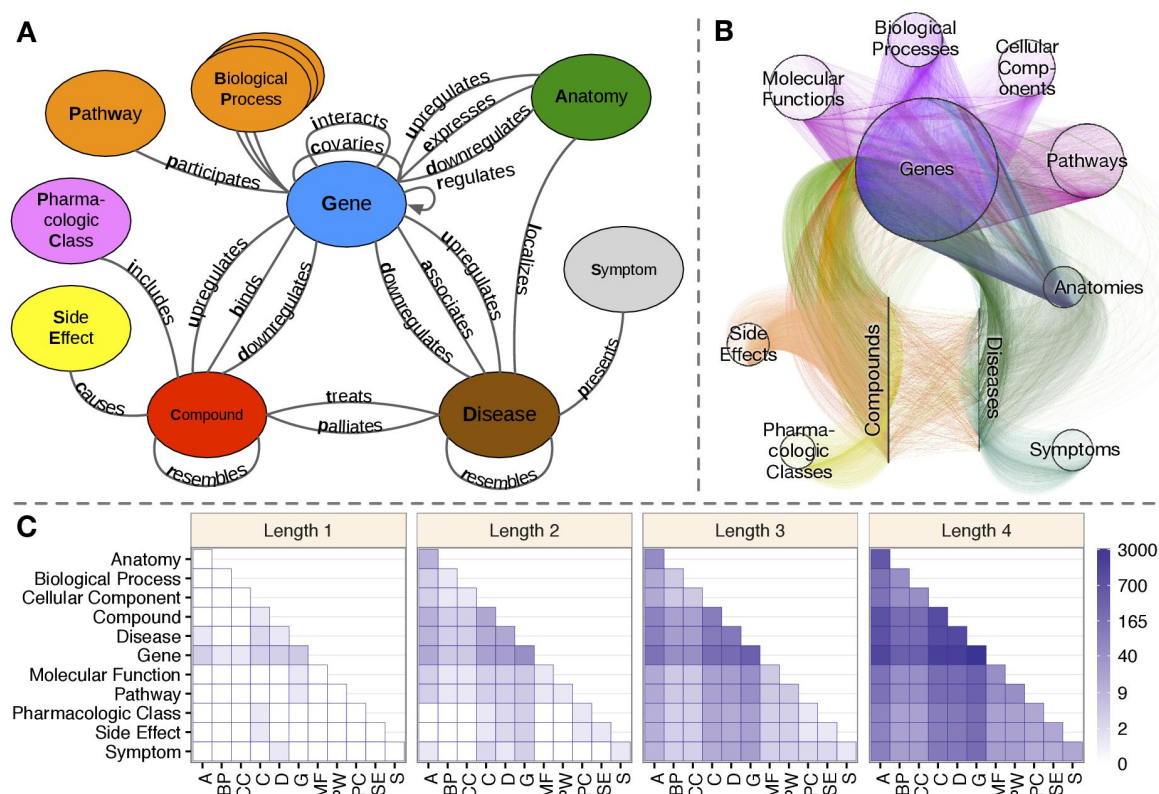


Figure 4.3. Hetionet knowledge graph

5. Main Framework

A causal knowledge graph is not merely a static record of the literature: it can be systematically audited, weighted, and enriched with observational data, and subsequently used as the basis for explainable clinical prediction of drug-related outcomes. This thesis operationalizes that idea through two complementary graph neural network tasks, applied to a shared causal knowledge graph of pharmacotherapy:

- Task A:** link prediction, which reconstructs and updates the knowledge graph based on real patient data;
- Task B:** node classification, which predicts patient-specific clinical endpoints based on the knowledge graph and real patient data.

Since both tasks operate on the same underlying knowledge graph, we adopt graph neural networks as the common modeling framework, and enrich them with Hierarchical Correlation Reconstruction (HCR). In fig. 5.1 there is presented high-level approach to construct GNN with HCR. While the two tasks share this common foundation, their implementation details differ in several respects; we therefore present each task separately in the following subsections.

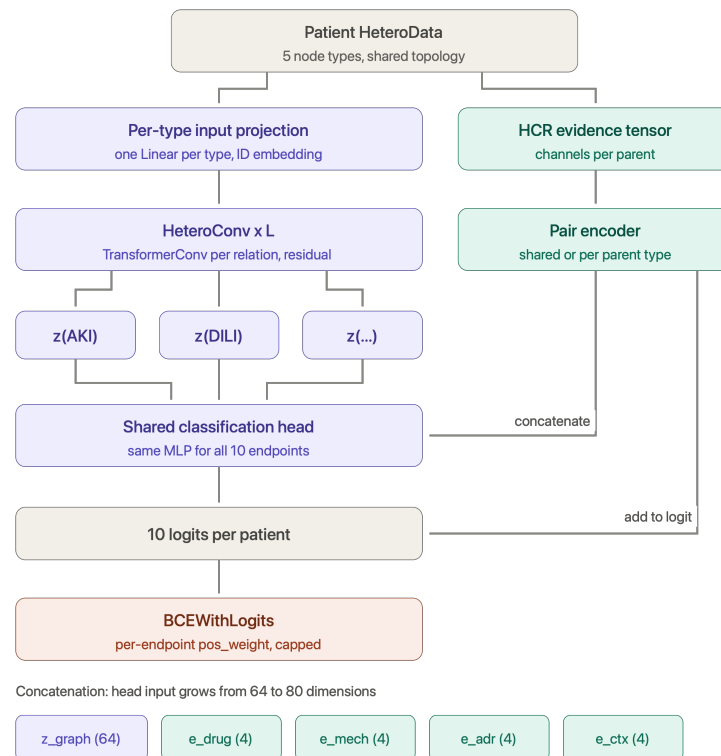


Figure 5.1. The presentation of approach to node classification task.

5.1. Link prediction in Graph Reconstruction

5.2. Endpoint prediction

Task B is formulated as multi-label node classification on a heterogeneous, per-patient graph. Rather than building one global graph and treating patients as additional nodes, every patient is represented as a copy of the same fixed causal DAG topology, with node feature vectors instantiated from that patient's individual covariates. The prediction targets are a fixed subset of clinical events on patient site. Formally, for a patient graph $G_p = (V, E)$ shared across the cohort, with patient-specific node features x_p , the model computes

$$\hat{y}_{p,e} = \sigma(f_\theta(G_p, x_p)_e), \quad e \in \mathcal{E}_{\text{target}}$$

where f_θ is a heterogeneous graph neural network shared across all patients, and only the leaf-level target embeddings are read out by the classification head.

5.2.1. Data Representation

The first thing in applying GNNs is to build properly the input into neural network. The input graph for the node classification task was built based on:

- a) **nodes**
- b) **edges**
- c) **per-scenario patient sample table**

Based on those items HeteroData object was created which was represented by objects type presented in the table

The patient data is encoded on nodes level. In fig. 5.1 there is presented the exemplary DAG for one of patient.

Two structural exclusion mechanisms are applied before any feature is computed:

- a) nodes flagged `exclude_from_observed_model` or `is_latent` in the audit (hidden confounders);
- b) nodes that are structural descendants of any endpoint in the DAG (e.g. documentation nodes such as `hospital_contact`, `fall_reported`, which are caused by the endpoint rather than causal parents of it)

are detected by graph descendant traversal and excluded from the feature set, even though they are not explicitly flagged in the source data. All normalization statistics (mean/std for continuous node values) and all HCR evidence statistics (section 5.2.3) are computed exclusively on the training split of patients, to prevent both node-level and patient-level information leakage across splits.

Tabela 5.1. Components of the per-patient HeteroData graph representation.

Component	Description
Node types	Six semantic categories of nodes shared across all patient graphs: <code>patient_context</code> , <code>drug_exposure</code> , <code>mechanism</code> , <code>adr_or_intermediate_state</code> , <code>observation_or_selection</code> , and <code>clinical_endpoint</code> .
Relations	Keyed by (<code>src_type</code> , "to", <code>dst_type</code>) weighted by <code>edge_attr</code> instead of using <code>edge_type</code> . This keeps the number of distinct convolutional relation modules in HeteroConv small while still letting edge-type information reach the message function.
Edge attributes (<code>edge_attr</code>)	A $1 + 3 + \text{edge types} $ -dimensional vector per relation: signed effect size, activation frequency, mechanistic confidence, evidence weight, and the edge-type one-hot. Convolutions consuming a scalar <code>edge_weight</code> (GraphConv/GCN) receive only the signed effect-size column; convolutions consuming a full <code>edge_attr</code> (GAT, Transformer) receive the whole vector.
Node features (<code>x</code>)	Static audited attributes plus a topological-layer encoding (numeric or onehot, configurable), combined with the patient's own observed value for that node where the node is visible under the current observability regime.

5.2.2. Model architecture

The core architecture is a stack of heterogeneous graph convolutional layers wrapped in HeteroConv, shared across relation types but parameterized identically for every patient graph. Four convolution backbones are supported through a common registry and were benchmarked against each other: GraphSAGE, GraphConv (GCN-style), GATv2, and TransformerConv. Because these operators were originally designed for homogeneous graphs, each is instantiated once per edge type inside HeteroConv, so the heterogeneous DAG is handled without collapsing node/edge types into a single homogeneous graph. As benefit of casual graph structure is possibility to encode deep structured path for node classification, multiple number of layers was tested to choose the best model.

A design issue that was handled during experiments is the duplication of the self-transform across relations. Several PyG convolution layers (SAGEConv, TransformerConv) internally add their own linear self-loop term, wrapped per-relation inside HeteroConv, this term is silently duplicated once per incoming relation type for a given node type. The implementation removes this internal duplication (`root_weight=False` for SAGE/Transformer) and replaces it with a single, explicit, controlled self-transform per layer per node type, applied once in the shared encoder rather than once per relation.

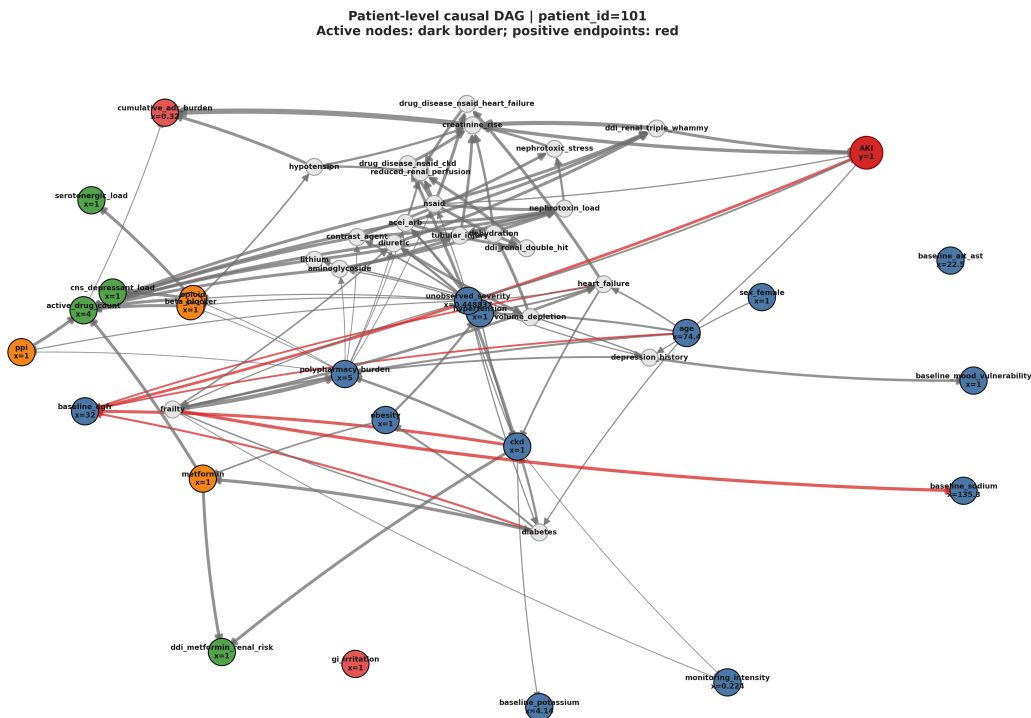


Figure 5.2. The description of the two tasks addressed in the thesis.

For GAT-family layers, where the self-dependency is baked into the attention query and cannot be disabled, the explicit self-transform acts as an additional controlled term on top of the irreducible built-in dependency. GraphConv/GCN, which has no `root_weight` parameter at all, is deliberately excluded from this mechanism, since its built-in self-transform cannot be removed without reimplementing the layer.

A node-identity embedding, added after the input projection, was introduced to resolve a specific representational collapse. Message-passing GNNs are bounded in expressive power by the 1-dimensional Weisfeiler-Lehman graph isomorphism test: two nodes with identical initial features and structurally equivalent neighborhoods cannot be assigned distinct representations by any number of convolutional layers. In the dataset, several endpoint nodes of the same type (e.g. Serotonin syndrome, Rhabdomyolysis, Lactic acidosis) share identical static audited attributes (e.g. `severity_weight`, `observability`) and comparable topological positions, and were therefore indistinguishable to the model except through such positional cues. To break this symmetry, a learnable embedding table is maintained per node type, indexed by each node's fixed local index, and added to the projected input features before the first convolutional layer, following the same principle underlying position aware node embeddings in .

This connects directly to the model's handling of oversmoothing. Using multiple message-passing layers risks oversmoothing the embedding signal, where repeated aggregation causes node representations to converge toward indistinguishability. To mitigate

this, several oversmoothing countermeasures were applied: residual connections, Jumping Knowledge, DropEdge, and PairNorm. These methods were evaluated both individually and in combination with the HCR enrichment described in section 5.2.3.

5.2.3. HCR application

In addition to the purely structural GNN pathway, approach presented in the thesis incorporates a statistical evidence side-channel - HCR described in details in section 3.2. This channel is motivated by the observation that some parent-endpoint dependencies are well captured by simple, well-understood pairwise statistics computed directly from the training cohort, and that combining such statistics with the deep GNN representation captures the strong signal from direct parents and smoothed signal from further layers. There were run couple of experiments with HCR configurations and GNNs. However, in general the approach was about computation of contingency-table statistics for each target endpoint and direct parents or pre-direct parents in the audited DAG (restricted to nodes with inferred binary, count, or continuous value types). The statistics computed was as followed per (parent, endpoint) pair: the φ -coefficient, normalized mutual information, a tanh-scaled log odds ratio, risk difference, and a tanh-scaled log lift, plus prevalence and support terms. There are nine to forty-one channels depending on configuration.

As these coefficients are computed with the patient's own endpoint label as part of the statistic, a naive computation would leak the label into the feature. This was addressed with an exact leave-one-out correction: every training patient's coefficient is computed from all other training patients only, using a closed-form re-weighting of the population statistic.

Continuous and count-valued parents are mapped to comparable pseudo-uniform scores via a train-only empirical CDF transform (with deterministic, patient-ID-seeded jitter for count variables), and Legendre polynomial bases are used to capture non-linear dependence on continuous parents.

Three integration modes were supported:

- a) linear approach - a single scalar HCR score added directly to the GNN logit,
- b) nonlinear encoder - parent-endpoint evidence vectors pooled and passed through a shared MLP
- c) separate encoders per parent node type - every direct parent passed through a separated MLP
- d) triple relations - relations not only focused on direct parents, but direct parents and grandparents.

The pooling in HCRPairEncoder masks out padding positions (endpoints with fewer parents than the batch's maximum) and returns a neutral zero vector for endpoints with no eligible parents, avoiding division by zero.

5.2.4. Training setup

Training iterates over patient graphs batched via PyTorch Geometric’s DataLoader. The experiment were run with targeted binary cross-entropy and focal loss with class-balance-aware weighting. Weight and focal alpha were computed from the training split’s endpoint prevalence, since adverse events are rare and highly imbalanced. Model selection uses ReduceLROnPlateau on validation performance, and evaluation is reported separately per endpoint and in aggregate, using AUROC, AUPRC, and prevalence-normalized lift, alongside the representation-level over-smoothing diagnostics (MAD, cosine similarity, Dirichlet energy) used to relate architectural choices. Due to rare events the main metric for model choice was AUPRC.

6. Experiments results

6.1. Link prediction in graph reconstuction

6.2. Endpoint prediction

6.2.1. Architecture

The first set of experiments established how much the choice of message-passing operator matters for this task. Three convolutions were compared: R-GCN, GraphSAGE and TransformerConv. All three are defined for homogeneous graphs, so each was wrapped in HeteroConv, which instantiates a separate convolution for every relation type and aggregates the results at the target node and allows to preserve the heterogenous structure.

All experiments share the same encoder structure and differ only in a small number of hyperparameters: the number of layers L , the presence of a residual connection and the HCR variant. Everything else was held fixed. The shared settings are listed in table 6.1, and the parameters specific to individual convolutions in table 6.2.

Tabela 6.1. Hyperparameters shared by all architectures. Values were fixed across the experiments reported in this chapter

Group	Hyperparameter	Value
Encoder	Hidden units per convolutional layer	64
	Number of layers L	1, 2, 3, 4, 5, 6
	Aggregation across relations	sum
	Activation	ReLU
	Batch normalisation	yes
Regularisation	Dropout	0.2
	Weight decay	5×10^{-4}
Classification head	Number of layers	2
	Hidden units	64 (equal to encoder width)
	Output units	1 per endpoint
Optimisation	Optimiser	Adam
	Learning rate	1×10^{-3}
	Batch size	128
	Epochs (maximum)	200
	Loss	weighted BCE with logits
	Positive weight cap	30
	Gradient clipping	disabled
	Seed	42
Model selection	Selection metric	validation AUPRC
	Early-stopping patience	20 epochs
	Minimum improvement	0.001

All three convolutions are wrapped in HeteroConv, which instantiates a separate convolution for each of the 19 relation types in the graph. The parameter counts therefore scale with the number of relations. In attention-based layers the hidden width is split across heads, so the output width stays at 64 regardless of the number of heads.

Tabela 6.2. Architecture-specific hyperparameters.

Hyperparameter	R-GCN	GraphSAGE	TransformerConv
Attention heads	—	—	4
Units per head	—	—	16
Output width per relation	64	64	$4 \times 16 = 64$
Basis decomposition	8 bases	—	—
Uses edge attributes	indirectly, via relation id	no	yes, via <code>edge_dim</code>
Weight matrices per relation	shared through bases	2	3 (query, key, value)

The results for those experiments are reported in table 6.3. All of metrics reported are described in section 3.2.1. The epoch is selected on the validation set and test metrics are reported from that epoch. The experiments assumed to used multiple number of layers as patient path is up to 6 degrees and it is assumed that every point can have an influence in casual path. For each experiment all number of layers were calculated, but for reporting reasons in table 6.3 there are included layers 1,3 and 6 to show the trend with the increased number of layers. The convention is liked that unless the results from other layers are significant for conclusion.

The results of these experiments are reported in table 6.3. All reported metrics are defined in section 3.2.1. In every case the epoch is selected on the validation set and the test metrics are taken from that epoch.

The range of depths follows the assumption that the longest path from a drug exposure to an endpoint in the audited DAG spans up to six hops, and every node along such a path may contribute to the causal chain. Models with one to six layers were therefore trained for every configuration. For readability only $L \in \{1, 3, 6\}$ are listed in table 6.3, which is sufficient to show the trend. Intermediate depths are reported only where they are relevant to a specific conclusion.

Without residual connections the three architectures diverge sharply at $L=6$, collapse to constant predictions (AUC=0.50).

Tabela 6.3. Message-passing architectures without residual connections.

Architecture	L	Validation				Test			
		AUC	AUPRC	Lift	% ceil.	AUC	AUPRC	Lift	% ceil.
R-GCN	1	0.623	0.124	2.28	81.6	0.624	0.120	2.21	82.0
GraphSAGE	1	0.622	0.124	2.28	80.9	0.619	0.120	2.21	78.9
TransformerConv	1	0.622	0.125	2.30	81.2	0.620	0.120	2.20	79.6
R-GCN	3	0.542	0.076	1.40	28.0	0.542	0.068	1.26	27.7
GraphSAGE	3	0.549	0.079	1.45	32.2	0.549	0.072	1.32	32.8
TransformerConv	3	0.550	0.083	1.53	33.2	0.552	0.073	1.35	34.5
R-GCN	6	0.503	0.055	1.00	2.2	0.500	0.054	1.00	0.0
GraphSAGE	6	0.530	0.065	1.20	20.0	0.510	0.061	1.12	6.6
TransformerConv	6	0.527	0.065	1.20	18.0	0.500	0.054	1.00	0.0

That's why the next step was application of residual connections described in section 3.2.1 to see the capabilities of models with higher number of layers. The results are presented in table 6.4.

Tabela 6.4. The same architectures with residual connections.

Architecture	L	Validation				Test			
		AUC	AUPRC	Lift	% ceil.	AUC	AUPRC	Lift	% ceil.
R-GCN	1	0.625	0.125	2.30	82.7	0.625	0.121	2.22	83.2
GraphSAGE	1	0.621	0.124	2.27	80.6	0.619	0.120	2.21	78.7
TransformerConv	1	0.623	0.126	2.31	81.3	0.619	0.120	2.21	79.0
R-GCN	3	0.630	0.133	2.44	86.0	0.627	0.125	2.29	84.2
GraphSAGE	3	0.623	0.132	2.42	81.4	0.616	0.124	2.27	77.0
TransformerConv	3	0.628	0.135	2.47	85.2	0.620	0.123	2.27	79.7
R-GCN	6	0.629	0.135	2.48	85.6	0.626	0.123	2.27	83.9
GraphSAGE	6	0.628	0.134	2.46	84.9	0.624	0.127	2.33	82.3
TransformerConv	6	0.627	0.134	2.46	84.2	0.620	0.122	2.24	79.4

With residual connections all three architectures converge to comparable performance (table 6.4): 77–86% of the Bayes ceiling on the test set, with differences of about 0.01 AUC. The more informative factors for model assessment are AUPRC and Lift, which show that the model performs nearly 2.3 times better than a random baseline. Since the architectures achieve comparable performance, the choice for the remaining experiments was therefore made on structural grounds. TransformerConv is the only one of the three that conditions message passing on edge attributes, which the audited DAG provides (mechanistic confidence, evidence weight, effect size, and sign). Since the role of edge-level evidence

is important for causal classification, TransformerConv was selected for all subsequent experiments with HCR.

6.2.2. HCR configuration

HCR was introduced to supply the model with statistical evidence about grandparent/parent–endpoint dependence that the message-passing path has to infer on its own (section 5.2.3). The configurations examined here vary along three axes, each addressing a separate question. The first is the form of the wide branch: a single learned coefficient per endpoint applied to a precomputed dependence score, against a small nonlinear encoder applied to each parent separately. The second is the scope of the evidence: binary parents only, or all node types, in which case continuous and count variables are expanded in a Legendre basis. The third is the order of the statistic: pairwise coefficients between a parent and an endpoint, or third-order moments over grandparent–parent–endpoint paths, which capture interactions that pairwise coefficients cannot represent. The relevant configurations are listed in table 6.6 – the following abbreviation applies to a configuration type.

Tabela 6.5. Notation used for the HCR configurations in the result tables.

Label	Join point	Description
—	—	No HCR branch
linear	sum at logit	Precomputed dependence score, all parent types; one learned scalar per endpoint, no encoder
nonlin. all 42D	sum at logit	42 evidence channels, all parent types; shared pair encoder
nonlin. bin. 10D	sum at logit	10 evidence channels, binary parents only; shared pair encoder
nonlin. bin. 42D	sum at logit	42 evidence channels, binary parents only; shared pair encoder
triple	sum at logit	13 channels of third-order moments over grandparent–parent–endpoint paths, replacing the pairwise evidence
typed+triple	concatenation	42 pairwise channels split across four non-shared encoders, one per parent type, each with a learned per-endpoint gate, plus an ungated third-order block

Without HCR the model degrades sharply with depth and predicts a constant at $L = 6$. Every HCR variant recovers a substantial part of the lost performance, and the gain grows with depth: +1.8, +42.5 and +74.5 percentage points of the Bayes ceiling at $L = 1, 3$ and 6 respectively. The concatenated variant is the only one that remains flat across all six depths.

The reason for improved results with HCR is that HCR branch reads the dependence statistics of the direct parents and grandparents from a precomputed tensor and never passes through a graph convolution, so its contribution is identical at $L = 1$ and at $L = 6$.

Tabela 6.6. HCR variants on TransformerConv *without* residual connections.

L	HCR	Validation				Test			
		AUC	AUPRC	Lift	% ceil.	AUC	AUPRC	Lift	% ceil.
1	—	0.622	0.1251	2.30	81.19	0.620	0.1197	2.20	79.61
1	linear 42D	0.622	0.1247	2.29	81.10	0.617	0.1191	2.19	77.83
1	nonlin. all 42D	0.622	0.1243	2.28	81.17	0.618	0.1192	2.19	78.57
1	typed+triple	0.634	0.1348	2.48	89.05	0.626	0.1241	2.28	83.46
2	nonlin. all 42D	0.625	0.1242	2.28	82.77	0.615	0.1180	2.17	76.15
2	typed+triple	0.628	0.1328	2.44	84.91	0.621	0.1235	2.27	80.16
3	—	0.550	0.0833	1.53	33.19	0.552	0.0734	1.35	34.49
3	linear 42D	0.604	0.1173	2.15	69.28	0.612	0.1031	1.90	74.51
3	triple	0.602	0.1131	2.08	67.92	0.590	0.1021	1.88	59.81
3	nonlin. all 42D	0.621	0.1310	2.41	80.13	0.629	0.1197	2.20	85.63
3	typed+triple	0.625	0.1362	2.50	82.90	0.620	0.1252	2.30	79.54
4	nonlin. all 42D	0.616	0.1214	2.23	76.90	0.609	0.1168	2.15	72.30
4	typed+triple	0.631	0.1350	2.48	86.65	0.614	0.1237	2.27	75.53
5	nonlin. all 42D	0.617	0.1183	2.17	77.67	0.611	0.1132	2.08	73.37
5	typed+triple	0.627	0.1320	2.42	84.31	0.617	0.1225	2.25	77.58
6	—	0.527	0.0653	1.20	17.96	0.500	0.0544	1.00	0.00
6	linear 42D	0.616	0.1206	2.21	76.91	0.603	0.0955	1.76	68.22
6	triple	0.607	0.1154	2.12	70.71	0.579	0.0967	1.78	52.56
6	nonlin. all 42D	0.569	0.0955	1.75	45.53	0.500	0.0544	1.00	0.00
6	typed+triple	0.630	0.1326	2.43	86.53	0.618	0.1218	2.24	78.35

Residual connections address the same failure from the opposite side: they preserve the signal inside the propagation channel instead of bypassing it. Based on results of experiments, we can observed that both are effective, but they are not complementary. Comparing models with enabled residual connections, no HCR variant improves on the model without HCR (table 6.7), and combining the two does not yield models alone. Once the propagation path with residual connections is repaired HCR does not provide any contribution.

Tabela 6.7. HCR variants on TransformerConv *with* residual connections.

L	HCR	Validation				Test			
		AUC	AUPRC	Lift	% ceil.	AUC	AUPRC	Lift	% ceil.
1	—	0.623	0.1255	2.31	81.30	0.619	0.1202	2.21	79.02
1	nonlin. bin.	0.622	0.1238	2.27	81.12	0.617	0.1176	2.16	77.82
1	nonlin. 42D	0.621	0.1250	2.30	80.56	0.616	0.1186	2.18	76.95
1	typed+triple	0.631	0.1335	2.45	86.94	0.621	0.1240	2.28	80.30
2	typed+triple	0.631	0.1322	2.43	86.77	0.619	0.1230	2.26	79.05
3	—	0.628	0.1346	2.47	85.23	0.620	0.1233	2.27	79.66
3	nonlin. bin.	0.627	0.1339	2.46	84.06	0.617	0.1242	2.28	77.48
3	nonlin. 42D	0.630	0.1343	2.47	85.95	0.619	0.1247	2.29	78.80
3	typed+triple	0.629	0.1350	2.48	85.64	0.618	0.1246	2.29	78.05
4	typed+triple	0.627	0.1367	2.51	84.10	0.621	0.1256	2.31	80.08
5	typed+triple	0.626	0.1361	2.50	83.88	0.623	0.1256	2.31	81.54
6	—	0.627	0.1337	2.46	84.19	0.620	0.1219	2.24	79.37
6	nonlin. bin.	0.614	0.1177	2.16	75.90	0.605	0.1082	1.99	69.84
6	nonlin. 42D	0.624	0.1301	2.39	82.21	0.618	0.1214	2.23	78.23
6	typed+triple	0.629	0.1347	2.47	85.62	0.620	0.1244	2.29	79.34

For understanding how HCR branch contributes to individual endpoints prediction, the learned gate weights are shown in fig. 6.1. Each cell gives the scalar by which the model scales the evidence block of one parent type before it enters the classification head; the gates were initialised at 1.0, so values below unity indicate reduced reliance on that channel.

Averaged over endpoints the gates order as *adr_or_intermediate_state* (0.72), *patient_context* (0.38), *mechanism* (0.18) and *drug_exposure* (0.00), which is the same order as the number of endpoints that possess a parent of each type (11, 8, 3 and 2 out of 11). The model therefore recovers the layout of the causal graph from the gradient alone.

To sum up, it is worth to emphasize that all of models with HCR reached a satisfactory results comparable to models with residual connections applied. Taking into consideration gate matrix, it could be observed that HCR brings value to specific number of endpoints which corresponds with higher prediction accuracy. In general the value of the HCR branch therefore does not lie in raising the peak, but in bringing stability to learning across all depths and additional information valid for the clinical event prediction. Across the six depths evaluated, *typed+triple* is the only variant that never collapses, retaining between 82% and 89% of the ceiling, while both the baseline and *nonlin. all 42D* fall below 80.00% at $L = 6$. The configuration *typed+triple* is mostly stable on depth and additionally bring the most informative value. The highest validation score in table 6.6 is obtained

6. Experiments results

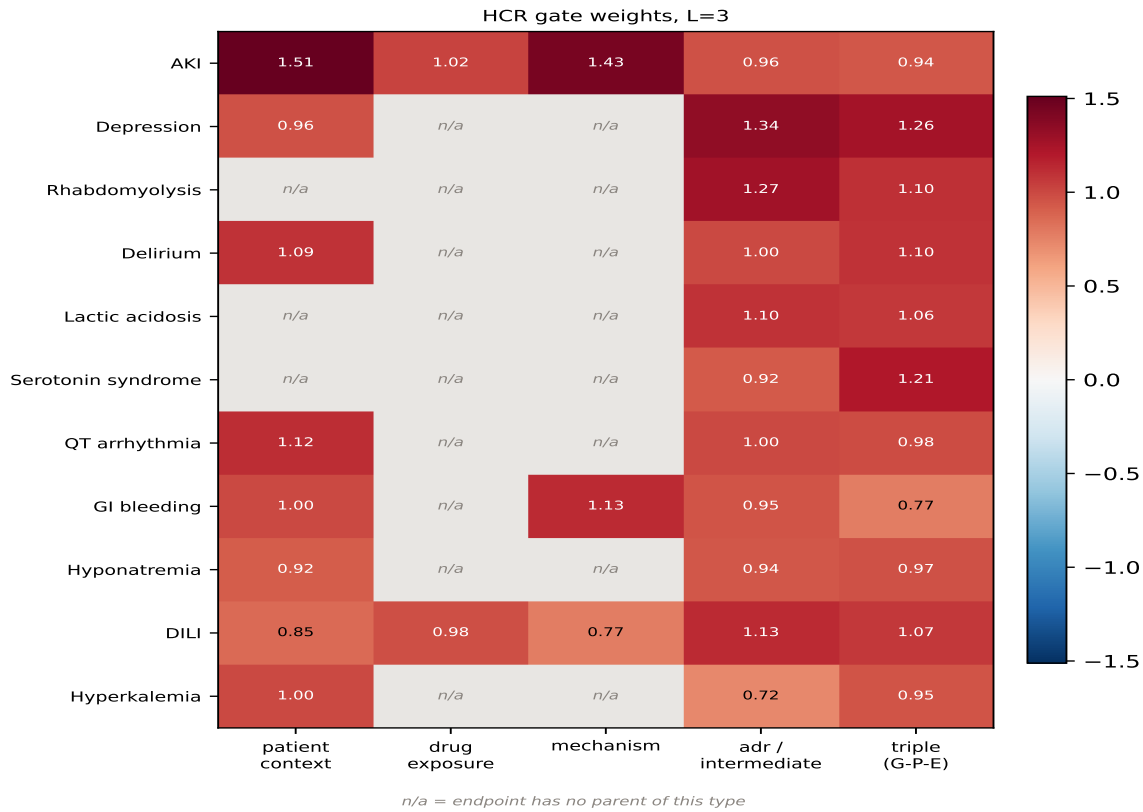


Figure 6.1. HCR gate matrix - influence of nodes type per endpoint

by *typed+triple* at $L = 1$ and $L = 4$ (89.05% and 86.65% of the Bayes ceiling). Bringing all together *typed+triple* HCR variant is chosen as the best.

6.2.3. Results per endpoint

Endpoints are ordered by prevalence. % ceil. is $(\text{AUC} - 0.5) / (\text{ceiling} - 0.5)$ against the per-endpoint Bayes ceiling; values above 100% indicate that the estimate on a finite sample exceeds the ceiling computed on the generator, and should be read as saturation rather than as exceeding the theoretical limit.

Tabela 6.8. Per-endpoint results of the *typed+triple* configuration at $L = 4$ without residual connections.

Endpoint	Prev.	Ceiling		Validation			Test		
		AUC	AUPRC	AUC	AUPRC	% ceil.	AUC	AUPRC	% ceil.
AKI	0.131	0.811	0.454	0.794	0.406	94.3	0.785	0.421	91.6
Depression	0.085	0.652	0.166	0.655	0.165	101.6	0.639	0.170	91.0
Hyponatremia	0.079	0.638	0.168	0.613	0.132	81.5	0.607	0.142	77.5
Hyperkalemia	0.049	0.590	0.083	0.595	0.080	105.7	0.550	0.074	55.2
Delirium	0.037	0.668	0.100	0.649	0.093	89.0	0.687	0.098	111.4
GI bleeding	0.021	0.570	0.080	0.570	0.080	99.0	0.528	0.029	39.6
DILI	0.018	0.653	0.086	0.563	0.067	41.3	0.579	0.036	51.6
QT arrhythmia	0.016	0.622	0.075	0.591	0.050	74.1	0.536	0.019	29.4
Macro	0.055	0.651	0.151	0.629	0.134	85.4	0.614	0.124	75.5
<i>Trained but excluded from the macro average</i>									
Lactic acidosis	0.0057	0.523	0.011	0.560	0.009	–	0.549	0.008	–
Rhabdomyolysis	0.0037	0.530	0.011	0.536	0.005	–	0.724	0.033	–
Serotonin syndrome	0.0027	0.554	0.008	0.646	0.010	–	0.642	0.008	–

7. Models benchmark

results for proxy dataset

7.1. Link prediction

7.2. Endpoint prediction

7.2.1. Data Representation

The benchmark dataset for endpoint prediction is Hetionet knowledge graph described in section 4.2.2. The structure of dataset is similar to synthetic dataset - heterogeneous with multi-label endpoints, that's why it was the choice for running benchmark on it. Even though the structure is similar, it was necessary to adjust it to node classification task similarly as synthetic dataset. The following adjustment was made to run benchmark experiment.

The benchmark for endpoint prediction is the Hetionet knowledge graph described in section 4.2.2. Its structure resembles the synthetic dataset — heterogeneous, with multi-label targets — which is why it was selected. Adapting it to node classification nevertheless required reconstruction listed below.

- (i) **Instance.** A compound takes the role of the patient: the topology is shared and a compound is described by the edges linking it to the remaining entities. A compound is therefore *not* a node of the graph, which means an entity can serve as a feature column only if it connects to compounds directly.
- (ii) **Target.** The predicted endpoint is a disease node, labelled positive when a *treats* or *palliates* edge links the compound to that disease (the two relations were merged). Prevalences range from 1.29% to 4.51% over the twenty most frequent diseases.
- (iii) **Layer mapping.** Node types were mapped onto the slots of the synthetic schema as presented in table 7.1.
- (iv) **Direction.** Hetionet is largely undirected and carries no causal orientation. So every edge was oriented from the lower to the higher layer and intra-layer edges were discarded, which guarantees acyclicity. The direction is a construction decision to check the model.
- (v) **Path length.** Two changes lengthen the paths to a disease. First, a pathway and biological-process layer was inserted as the third layer, while the direct DaG edges were kept. Second, class-to-gene edges were added, because pharmacologic classes connect only to compounds in Hetionet and would otherwise have no edges at all. Such an edge is created when at least three training compounds of a class bind the same gene.

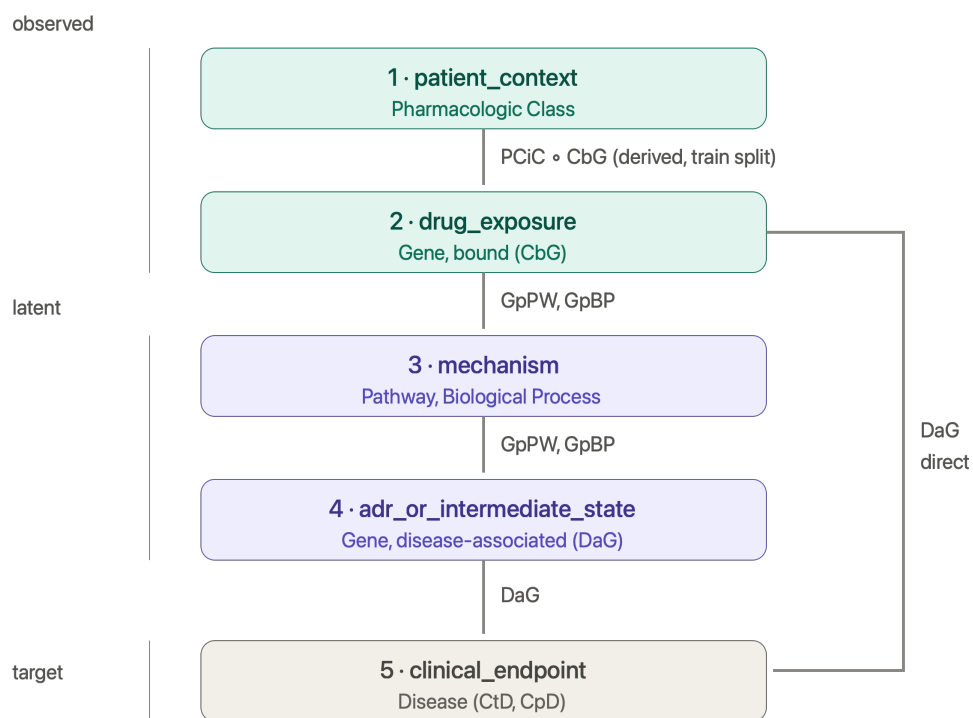
In fig. 7.1 there is presented the updated view of graph.

7.2.2. Results

The proxy dataset reproduces the central pattern of the synthetic experiments, but with one difference that is worth stating carefully. The gain from HCR is again conditional on the state of the propagation path: at $L = 6$ without residual connections the baseline collapses to 0.585 AUC on test, while every HCR variant recovers a substantial part of it, up

Tabela 7.1. Mapping of Hetionet entities onto the synthetic layer schema.

Layer	Slot	Hetionet entity	Observed
1	patient_context	Pharmacologic Class (filtered)	yes
2	drug_exposure	Gene, bound (CbG)	yes
3	mechanism	Pathway, Biological Process	no (latent)
4	adr_or_intermediate_state	Gene, associated (DaG)	no (latent)
5	clinical_endpoint	Disease	target

**Figure 7.1.** HCR gate matrix - influence of nodes type per endpoint

to 0.757 for *typed+triple*. At $L = 1$, where the receptive field of an endpoint coincides with the input of the HCR branch, no variant improves on the baseline, and the concatenated variants are slightly worse.

The difference concerns the interaction between HCR and residual connections. On the synthetic data the two mechanisms did not add up: once the residual path restored the flow of information through depth, no HCR variant improved on the model without it. On the proxy dataset they do combine at $L = 6$: the baseline reaches 0.585 without either mechanism, 0.680–0.757 with HCR alone, 0.728 with residual alone, and 0.760 with both. The combination is therefore better than either component in isolation.

A structural explanation is available. In the synthetic graph the direct parents of an endpoint are observed, so the HCR branch and the one-hop receptive field read the same variables and the two mechanisms address the same bottleneck. In the Hetionet-derived graph the parents that carry per-compound values sit in layer 2, while layers 3 and 4 are latent relays; information must therefore traverse several hops, and the residual path and the HCR bypass contribute at different points of that route. This is consistent with the depth curve on the proxy dataset, which rises monotonically with L for *typed+triple* without residual connections (0.662, 0.677, 0.702, 0.757 at $L = 1, 2, 3, 6$) — the opposite of the flat curve observed on the synthetic data.

The comparison is limited by unequal coverage: *linear* and *nonlin. all* were only run at $L = 3$, and several depths are missing for individual variants. Differences below roughly 0.03 AUC fall within the uncertainty of a single seed at 310 validation compounds.

Tabela 7.2. Results under full observability on the benchmark dataset. *nonlin. bin.* uses a 9-channel evidence vector restricted to binary parents; *nonlin. all* uses 42 channels with the full parent scope; *typed+triple* concatenates one encoder per parent type with a third-order block. Coverage of depths differs between variants.

<i>L</i>	Res.	HCR	Validation			Test		
			AUC	AUPRC	Lift	AUC	AUPRC	Lift
1	no	—	0.757	0.188	7.92	0.664	0.120	6.49
1	no	nonlin. bin.	0.763	0.207	8.75	0.657	0.120	6.47
1	no	nonlin. all	0.761	0.206	8.69	0.665	0.140	7.55
1	no	typed+triple	0.765	0.160	6.75	0.662	0.104	5.63
1	yes	—	0.756	0.196	8.26	0.669	0.106	5.72
1	yes	nonlin. bin.	0.756	0.201	8.49	0.670	0.135	7.31
1	yes	nonlin. all	0.758	0.206	8.69	0.664	0.135	7.32
1	yes	typed+triple	0.750	0.157	6.61	0.661	0.098	5.30
2	no	nonlin. all	0.763	0.263	11.10	0.694	0.173	9.34
2	no	typed+triple	0.778	0.263	11.10	0.677	0.175	9.46
2	yes	nonlin. all	0.760	0.224	9.44	0.684	0.154	8.31
2	yes	typed+triple	0.765	0.232	9.79	0.690	0.174	9.43
3	no	—	0.756	0.199	8.38	0.722	0.152	8.24
3	no	nonlin. bin.	0.805	0.275	11.62	0.686	0.169	9.14
3	no	nonlin. all	0.800	0.259	10.91	0.700	0.166	8.99
3	no	typed+triple	0.794	0.237	10.01	0.702	0.168	9.11
3	yes	—	0.799	0.225	9.48	0.685	0.166	8.97
3	yes	nonlin. bin.	0.814	0.235	9.90	0.714	0.183	9.92
3	yes	nonlin. all	0.807	0.259	10.91	0.677	0.174	9.39
3	yes	typed+triple	0.805	0.248	10.45	0.708	0.180	9.72
4	no	nonlin. all	0.757	0.230	9.70	0.678	0.151	8.14
4	no	typed+triple	0.807	0.262	11.06	0.747	0.181	9.79
4	yes	nonlin. all	0.814	0.238	10.05	0.715	0.180	9.76
4	yes	typed+triple	0.806	0.240	10.12	0.713	0.168	9.06
5	no	nonlin. all	0.783	0.239	10.08	0.680	0.158	8.52
5	no	typed+triple	0.807	0.246	10.37	0.746	0.163	8.84
5	yes	nonlin. all	0.815	0.291	12.29	0.727	0.179	9.71
5	yes	typed+triple	0.813	0.270	11.37	0.724	0.211	11.41
6	no	—	0.610	0.121	5.11	0.585	0.077	4.14
6	no	nonlin. bin.	0.782	0.210	8.86	0.680	0.174	9.42
6	no	nonlin. all	0.794	0.237	9.99	0.682	0.157	8.51
6	no	typed+triple	0.803	0.263	11.11	0.757	0.175	9.45
6	yes	—	0.817	0.255	10.77	0.728	0.173	9.36
6	yes	nonlin. bin.	0.832	0.310	13.06	0.760	0.179	9.70
6	yes	nonlin. all	0.812	0.235	9.92	0.734	0.166	8.96
6	yes	typed+triple	0.808	0.251	10.59	0.730	0.205	11.10

8. Conclusions

Podsumowanie i krytyczna analiza osiągniętych rezultatów. Ocena, czy osiągnięto założony cel pracy. Dyskusja co można było zrobić lepiej i propozycja dalszych prac w celu usprawniania opracowanego rozwiązania.

9. Appendix

Variable classes and generation mechanisms

Tabela 9.1. Variable classes in the primary synthetic patient-level dataset.

Variable class	Number	Value type	Generation mechanism and role
patient_id	1	Integer	Unique technical patient identifier; not a graph node and not used as a predictive feature.
hospital_id	1	Categorical integer	Technical identifier of a simulated hospital. The primary clean scenario contains one hospital; multiple hospitals are used only in the multihospital extension.
patient_context	24	Mostly binary; selected count or baseline-measurement variables	Patient demographics, comorbidities, baseline clinical characteristics, latent severity, and treatment propensity. Values are initialised before downstream graph variables.
drug_exposure	29	Binary	Stochastic indicators of exposure to a drug class. Their probabilities are generated by the additive logistic model conditional on their direct graph parents.
mechanism	49	Binary, count, and deterministic gate variables	Physiological mechanisms, interaction gates, drug-load variables, and other mediating concepts. Values are either sampled by the additive logistic model or calculated deterministically from their components.

Continued on next page

Variable class	Number	Value type	Generation mechanism and role
adr_or_intermediate ²⁸ state	28	Mostly binary; one derived score and one threshold variable	Intermediate adverse states and physiological consequences. Most are generated stochastically; cumulative_adr_burden and severe_adr_burden are deterministic derived variables.
observation_or_selection	12	Binary	Variables representing testing, recording, reporting, or selection processes rather than the occurrence of a clinical state.
clinical_endpoint	13	Binary	Downstream clinical outcomes. Each is sampled from a Bernoulli distribution using a patient-specific probability conditional on its direct parents. These variables form the target labels for endpoint prediction.

Deterministic variables

Tabela 9.2. Deterministic variables and their generating rules.

Variable	Rule type	Generating rule
active_drug_count	Drug count	Sum of all 29 binary drug_exposure variables.
ddi_renal_double_hit	Conjunctive gate	nsaid $\wedge$ acei_arb.
ddi_renal_triple_whammy	Conjunctive gate	nsaid $\wedge$ acei_arb $\wedge$ diuretic.
ddi_cns_depression_synergy	Conjunctive gate	opioid $\wedge$ benzodiazepine.

Continued on next page

Variable	Rule type	Generating rule
ddi_bleeding_dual	Conjunctive gate	anticoagulant $\wedge$ antiplatelet.
ddi_bleeding_triple	Conjunctive gate	nsaid $\wedge$ anticoagulant $\wedge$ antiplatelet.
ddi_serotonergic_synergy	Conjunctive gate	ssri $\wedge$ snri.
ddi_hepatic_triple_hit	Conjunctive gate	hepatotoxic_drug_A $\wedge$ antibiotic_hepatic_risk $\wedge$ valproate_like_drug.
ddi_qt_multidrug_load	Drug count	qt_prolonging_drug + ssri + snri + macrolide + azole_antifungal.
nephrotoxin_load	Drug count	nsaid + aminoglycoside + contrast_agent + lithium + acei_arb.
hepatic_drug_load	Drug count	hepatotoxic_drug_A + antibiotic_hepatic_risk + valproate_like_drug + statin + macrolide + azole_antifungal.
qt_drug_load_v3	Drug count	qt_prolonging_drug + ssri + snri + macrolide + azole_antifungal + digoxin.
cns_depressant_load	Drug count	opioid + benzodiazepine + anticholinergic + valproate_like_drug.
bleeding_risk_load	Drug count	anticoagulant + antiplatelet + nsaid + ssri + corticosteroid.
serotonergic_load	Drug count	ssri + snri + opioid + triptan.
cyp_inhibitor_load	Drug count	macrolide + azole_antifungal.
ddi_statin_cyp_inhibitor	Conjunctive gate	statin $\wedge$ $\mathbb{1}(\text{cyp_inhibitor_load} > 0)$.
ddi_metformin_renal_risk	Conjunctive gate	metformin $\wedge$ ckd.

Continued on next page

Variable	Rule type	Generating rule
ddi_lithium_renal_risk	Conjunctive gate	$\text{lithium} \wedge \text{ckd}.$
ddi_digoxin_electrolyte_risk	Conjunctive gate	$\text{digoxin} \wedge \text{electrolyte_disturbance}.$
drug_disease_nsaid_ckd	Conjunctive gate	$\text{nsaid} \wedge \text{ckd}.$
drug_disease_nsaid_heart_failure	Conjunctive gate	$\text{nsaid} \wedge \text{heart_failure}.$
drug_disease_qt_baseline_risk	Conjunctive gate	$\mathbb{1}(\text{qt_drug_load_v3} > 0) \wedge \text{baseline_qt_risk}.$
drug_disease_hepatic_liver_disease	Conjunctive gate	$\mathbb{1}(\text{hepatic_drug_load} > 0) \wedge \text{liver_disease}.$
drug_disease_anticoagulant_frailty	Conjunctive gate	$\text{anticoagulant} \wedge \text{frailty}.$
ddi_serotonergic_high_load	Threshold	$\mathbb{1}(\text{serotonergic_load} \geq 2).$
cumulative_adr_burden	Weighted sum	$\sum_k \omega_k \cdot \text{ADR}_k + 0.08 \cdot \text{active_drug_count},$ where ω_k is the fixed severity weight for adverse-state component k .
severe_adr_burden	Threshold	$\mathbb{1}(\text{cumulative_adr_burden} \geq 4.0).$
elevated_creatinine_recorded	Recorded-state rule	$\text{creatinine_rise} \wedge \text{creatinine_tested}.$
elevated_alt_ast_recorded	Recorded-state rule	$\text{alt_ast_rise} \wedge \text{lft_tested}.$
qt_recorded	Recorded-state rule	$\text{qt_prolongation_state} \wedge \text{ecg_performed}.$

References

- [1] S. Toonsi, P. N. Schofield, and R. Hoehndorf, *Causal knowledge graph analysis identifies adverse drug effects*, arXiv preprint arXiv:2505.06949, Preprint, not yet peer-reviewed, 2025.

Spis rysunków

3.1 Porównanie bloków rezydualnych wykorzystywanych w sieci Swin Transformer, ResNet i ConvNeXt.	12
4.1 Frequency of nodes types.	17
4.2 DAG visualization	19
4.3 Hetionet knowledge graph	24
5.1 The presentation of approach to node classification task.	25
5.2 The description of the two tasks addressed in the thesis.	28
6.1 HCR gate matrix - influence of nodes type per endpoint	38
7.1 HCR gate matrix - influence of nodes type per endpoint	42

Spis tabel

4.1 Node types in the patient causal DAG.	17
4.2 Semantic categories of directed edges in the patient causal DAG.	18
4.3 Structural statistics of the causal knowledge graph.	18
4.4 Deterministic variables in the synthetic patient generator.	21
4.5 Synthetic evaluation scenarios and their intended data challenge.	22
4.6 Schema of the generated patient-level dataset.	23
4.7 Endpoint prevalence and AUROC ceiling in the evaluation subset.	23
5.1 Components of the per-patient HeteroData graph representation.	27
6.1 Hyperparameters shared by all architectures. Values were fixed across the experiments reported in this chapter	32
6.2 Architecture-specific hyperparameters.	33
6.3 Message-passing architectures without residual connections.	34
6.4 The same architectures with residual connections.	34
6.5 Notation used for the HCR configurations in the result tables.	35
6.6 HCR variants on TransformerConv <i>without</i> residual connections.	36
6.7 HCR variants on TransformerConv <i>with</i> residual connections.	37
6.8 Per-endpoint results of the <i>typed+triple</i> configuration at $L = 4$ without residual connections.	39
7.1 Mapping of Hetionet entities onto the synthethic layer schema.	42
7.2 Results under full observability on the benchmark dataset. <i>nonlin. bin.</i> uses a 9-channel evidence vector restricted to binary parents; <i>nonlin. all</i> uses 42 channels with the full parent scope; <i>typed+triple</i> concatenates one encoder per parent type with a third-order block. Coverage of depths differs between variants.	44

9.1	Variable classes in the primary synthetic patient-level dataset.	47
9.2	Deterministic variables and their generating rules.	48